



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251172
Nama Lengkap	Ang, Joshua Alexander Hartono
Minggu ke / Materi	15 / Fungsi Rekursif

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pengertian Rekursif

Fungsi rekursif adalah fungsi yang dapat memanggil dirinya sendiri secara berulang-ulang di dalam program. Rekursif biasanya digunakan untuk menyelesaikan masalah yang memiliki pola yang berulang dan terstruktur. Seperti pada perhitungan matematika atau proses tertentu yang dilakukan berkali-kali. Walau terlihat sederhana, penggunaan rekursif harus dilakukan dengan hati-hati karena jika tidak memiliki kondisi berhenti, program bisa berjalan terus menerus dan menyebabkan hang atau error.

Di dalam fungsi rekursif sendiri terdapat dua bagian penting yang harus ada agar program dapat berjalan dengan benar. Bagian pertama adalah base case, yaitu kondisi yang digunakan untuk menghentikan proses rekursif sehingga fungsi tidak berjalan tanpa akhir. Bagian kedua adalah recursive case, yaitu bagian yang berisi pemanggilan fungsi itu sendiri secara berulang sampai mencapai base case. Kedua bagian tersebut saling berhubungan karena proses pengulangan akan terus dilakukan hingga kondisi base case terpenuhi.

Kelebihan dan Kekurangan

Fungsi rekursif memiliki beberapa kelebihan yang sering digunakan. Salah satu keunggulan utama dari fungsi rekursif adalah membuat kode program menjadi lebih singkat, rapi, dan terlihat elegan dibandingkan dengan menggunakan banyak perulangan atau loop panjang. Dengan rekursif, kita dapat menyelesaikan suatu masalah hanya dengan menggunakan beberapa baris saja karena fungsi cukup memanggil dirinya sendiri hingga kondisi tertentu terpenuhi. Selain itu, rekursif juga membantu memecah masalah yang besar menjadi lebih kecil sehingga alur penyelesaian dapat lebih mudah disusun secara bertahap. Penggunaan rekursif juga membuat logika program pada beberapa kasus menjadi lebih mudah dipahami karena prosesnya mengikuti pola yang teratur. Sehingga kita hanya perlu memikirkan bagaimana satu langkah proses bekerja lalu fungsi akan mengulangi langkah tersebut secara otomatis.

Walaupun memiliki banyak kelebihan, fungsi rekursif juga memiliki beberapa kekurangan yang perlu diperhatikan. Salah satu kelemahan dari fungsi ini yaitu penggunaan memori yang cukup besar karena setiap kali fungsi memanggil dirinya sendiri, sistem akan menyimpan hasil proses pada memori sementara. Jika pemanggilan fungsi dilakukan terlalu banyak, maka memori yang digunakan akan semakin besar dan dapat menyebabkan program menjadi lambat. Selain itu, rekursif juga mengorbankan efisiensi dan kecepatan jika dibandingkan dengan perulangan biasa yang lebih sederhana. Hal ini karena rekursif kurang cocok pada program yang membutuhkan proses sangat cepat atau data dalam jumlah yang besar. Kelemahan lainnya adalah fungsi rekursif sering kali sulit untuk dipahami dan lebih rumit dalam melakukan debugging. Ketika terjadi error, kita harus menelusuri proses pemanggilan fungsi satu per satu karena fungsi berjalan secara berulang dan saling memanggil dirinya sendiri. Jika base case tidak dibuat dengan benar, program dapat mengalami infinite loop atau pemanggilan tanpa akhir yang menyebabkan program mengalami hang. Selain itu, bagi pemula konsep rekursif sering terasa membingungkan karena alur prosesnya tidak berjalan seperti program biasa yang dibaca dari atas ke bawah.

Bentuk Umum dan Studi Kasus

Bentuk umum fungsi rekursif pada Python adalah sebuah fungsi yang didalamnya terdapat pemanggilan terhadap dirinya sendiri. Pada fungsi rekursif biasanya terdapat parameter yang digunakan untuk menyimpan nilai yang akan diproses secara berulang. Walaupun terlihat berbeda, sebenarnya semua fungsi rekursif dibuat juga menggunakan cara iteratif atau perulangan biasa. Perbedaannya terletak pada cara penyelesaian masalahnya, dimana rekursif menggunakan pemanggilan fungsi secara terus-menerus sedangkan iteratif menggunakan looping for atau while. Oleh karena itu, rekursif sering digunakan untuk menyelesaikan masalah yang memiliki pola perhitungan bertahap dan berulang.

```
1  def function_name(parameter_list):  
2      ...  
3      function_name(...)  
4      ...
```

Gambar 15.1: Bentuk umum fungsi rekursif pada Python. Sumber: <https://shorturl.at/64Fki>

Salah satu contoh penggunaan rekursif adalah pada perhitungan faktorial. Faktorial merupakan proses perkalian berurutan dari angka 1 sampai dengan n . Pada cara iteratif, proses faktorial dilakukan menggunakan perulangan dari angka 1 hingga n sambil menyimpan hasil perkalian ke dalam variabel total. Dalam algoritma ini, setiap angka akan dikalikan satu per satu sampai seluruh proses selesai dilakukan. Namun pada rekursif faktorial dilakukan dengan menggunakan rumus faktorial yaitu $n \times \text{faktorial}(n-1)$. Rumus tersebut akan terus memanggil dirinya sendiri sampai mencapai batas base case. Pada contoh program faktorial, base case terjadi ketika nilai n sama dengan 0 atau sama dengan 1, karena hasil faktorial dari masing-masing angka 0 dan 1 adalah 1. Setelah mencapai kondisi tersebut, fungsi rekursif akan mengembalikan hasil dari perkalian secara bertahap hingga kembali ke pemanggilan pertama. Cara kerja seperti ini membuat proses faktorial terlihat lebih singkat karena perhitungan dilakukan melalui pemanggilan fungsi yang berulang.

$$\text{fact}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot \text{fact}(n - 1) & \text{if } n > 0 \end{cases}$$

Gambar 15.2: Rumus faktorial. Sumber: <https://shorturl.at/64Fki>

Pseudocode (recursive):

```
function factorial is:  
input: integer  $n$  such that  $n \geq 0$   
output:  $[n \times (n-1) \times (n-2) \times \dots \times 1]$   
  
    1. if  $n$  is 0, return 1  
    2. otherwise, return  $[n \times \text{factorial}(n-1)]$   
  
end factorial
```

Gambar 15.3: Pseudocode rekursif faktorial. Sumber: <https://shorturl.at/64Fki>

```
1  def faktorial(n):  
2      if n==0 or n==1:  
3          return 1  
4      else:  
5          return faktorial(n-1) * n  
6  
7  print(faktorial(4))
```

Gambar 15.4: Contoh program rekursif faktorial. Sumber: <https://shorturl.at/64Fki>

Kode program faktorial rekursif menggunakan fungsi bernama faktorial dengan parameter n . Jika nilai n adalah 0 atau 1 maka fungsi akan langsung mengembalikan nilai 1 menggunakan return. Namun jika nilai n ternyata lebih besar dari 1, maka fungsi akan memanggil dirinya sendiri dengan nilai $n-1$ lalu hasilnya akan dikalikan dengan n . Saat program menjalankan `print(faktorial(4))`, proses perhitungannya akan dimulai dari faktorial(4), lalu memanggil faktorial(3), kemudian faktorial(2), lalu faktorial(1) hingga mencapai base case. Proses pemanggilan fungsi secara bertahap tersebut menunjukkan bagaimana rekursif bekerja dari nilai terbesar menuju nilai yang paling kecil.

Setelah mencapai faktorial(1), proses perhitungan tidak berhenti melainkan mulai kembali dari proses sebelumnya sambil membawa hasil perhitungan. Nilai 1 akan dikembalikan oleh faktorial(2), lalu dihitung menjadi $2 * 1 = 2$. Setelah itu hasilnya akan dikembalikan lagi ke faktorial(3) menjadi $3 * 2 = 6$, kemudian ke faktorial(4) menjadi $4 * 6 = 24$. Alur pengembalian tersebut membuat hasil faktorial dapat dihitung secara berurutan sampai mendapatkan hasil yang diinginkan.

```
1.
2. calc_factorial(4)           # 1st call with 4
3. 4 * calc_factorial(3)      # 2nd call with 3
4. 4 * 3 * calc_factorial(2)  # 3rd call with 2
5. 4 * 3 * 2 * calc_factorial(1) # 4th call with 1
6. 4 * 3 * 2 * 1             # return from 4th call as number=1
7. 4 * 3 * 2                 # return from 3rd call
8. 4 * 6                     # return from 2nd call
9. 24                        # return from 1st call
```

Gambar 15.5: Proses perhitungan program rekursif faktorial. Sumber: <https://shorturl.at/64Fki>

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github : <https://github.com/angjoshua-sudo/Prak-Alpro-2026.git>

SOAL 1

Source code:

```
1 def prima(n, bagi = 2):
2     if n <= 1 :
3         return(f"{n} bukan bilangan prima")
4     if bagi == n :
5         return(f"{n} adalah bilangan prima")
6     if n % bagi == 0:
7         return(f"{n} bukan bilangan prima")
8
9     return prima(n, bagi+1)
10
11 n = int(input("Masukkan bilangan yang mau di cek prima atau bukan: "))
12 print(prima(n))
```

Penjelasan:

Pada bagian awal program, dibuat fungsi bernama **prima** yang digunakan untuk mengecek apakah sebuah bilangan termasuk bilangan prima atau bukan. Fungsi ini memiliki dua parameter yaitu **n** sebagai bilangan yang akan di periksa dan **bagi** sebagai angka pembagi yang secara otomatis dimulai dari 2. Program pertama-tama mengecek apakah nilai **n** kurang dari sama dengan 1 atau tidak. Hal ini dikarenakan angka tersebut bukan merupakan bilangan prima. Setelah itu, program akan memeriksa apakah nilai **bagi** sudah sama dengan **n**. Jika kondisi tercapai, berarti tidak ada angka lain yang dapat membagi habis bilangan tersebut sehingga program mengembalikan hasil bahwa bilangan tersebut adalah bilangan prima. Selanjutnya program akan menggunakan operasi modulus **n % bagi** untuk mengecek apakah bilangan dapat habis oleh angka tertentu. Jika hasil modulo sama dengan 0, maka program akan langsung mengembalikan hasil bahwa bilangan tersebut bukan bilangan prima. Namun, jika tidak habis dibagi, fungsi akan memanggil dirinya sendiri menggunakan **prima(n, bagi+1)** sehingga nilai pembagi terus bertambah satu per satu. Pada bagian akhir program, pengguna diminta untuk memasukkan angka melalui **input()**, lalu angka tersebut diubah menjadi integer. Hasil pengecekan kemudian ditampilkan di layar menggunakan **print(prima(n))**.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu15/soal1.py
Masukkan bilangan yang mau di cek prima atau bukan: 2
2 adalah bilangan prima
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu15/soal1.py
Masukkan bilangan yang mau di cek prima atau bukan: 9
9 bukan bilangan prima
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu15/soal1.py
Masukkan bilangan yang mau di cek prima atau bukan: 23
23 adalah bilangan prima
```

SOAL 2

Source code:

```
1 def polindrom(kalimat):
2     kalimat = kalimat.lower()
3     kalimat = kalimat.replace(' ','')
4
5     if len(kalimat) <=1:
6         return(f"Kalimat adalah polindrom")
7
8     if kalimat[0] != kalimat[-1]:
9         return(f"Kalimat bukan polindrom")
10
11     return polindrom(kalimat[1:-1])
12
13 kalimat = input("Masukkan kalimat: ")
14 print(polindrom(kalimat))
```

Penjelasan:

Pada bagian awal program, dibuat fungsi bernama **polindrom** dengan parameter **kalimat**. Pertama, program akan mengubah seluruh huruf menjadi huruf kecil menggunakan **lower()** agar perbedaan huruf kapital dan nonkapital tidak memengaruhi hasil. Setelah itu, spasi dalam kalimat dihapus menggunakan **replace(' ','')** sehingga kalimat dapat di periksa tanpa memperhatikan jarak antar kata. Selanjutnya program akan memeriksa apakah panjang dari **len(kalimat) <= 1**. Jika kondisi ini terpenuhi, maka kalimat akan dinyatakan sebagai polindrom karena semua huruf telah berhasil di periksa. Jika tidak, program akan membandingkan huruf pertama dengan huruf terakhir menggunakan **kalimat[0]** dan **kalimat[-1]**. Jika kedua huruf berbeda, maka program langsung mengembalikan hasil bahwa kalimat tersebut bukan polindrom. Namun, jika huruf pertama dan terakhir sama, maka program akan menjalankan rekursi dengan memanggil fungsi **polindrom(kalimat[1:-1])**. Pada bagian akhir, pengguna diminta memasukkan kalimat melalui **input()**, lalu hasil pemeriksaan ditampilkan menggunakan **print()**

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu15/soal2.py
Masukkan kalimat: pak karel
Kalimat bukan polindrom
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu15/soal2.py
Masukkan kalimat: kasur rusak
Kalimat adalah polindrom
```

SOAL 3

Source code:

```
1 def ganjil(angka):
2     if angka <= 0:
3         return 0
4
5     if angka % 2 == 0:
6         return ganjil(angka-1)
7
8     return angka + ganjil(angka-2)
9
10
11 angka = int(input("Masukkan angka: "))
12 print("Hasil deret ganjil adalah",ganjil(angka))
```

Penjelasan:

Pada bagian awal program, dibuat sebuah fungsi bernama **ganjil** dengan parameter **angka**. Program pertama-tama memeriksa apakah nilai **angka** kurang dari atau sama dengan 0 menggunakan **if angka <= 0**. Jika kondisi ini terpenuhi, fungsi akan mengembalikan nilai 0 karena tidak ada lagi bilangan ganjil yang perlu dijumlahkan. Setelah itu, program memeriksa apakah nilai **angka** merupakan bilangan genap atau tidak menggunakan operasi modulus **angka % 2 == 0**. Jika angka genap, maka fungsi akan memanggil dirinya sendiri dengan nilai **angka-1**. Jika **angka** adalah bilangan ganjil, maka program akan menjumlahkan nilai **angka** tersebut dengan hasil pemanggilan rekursif **ganjil(angka-2)**. Proses ini dilakukan hingga mencapai base case. Pada bagian akhir program, pengguna diminta memasukkan angka melalui **input()**, lalu input tersebut diubah menjadi tipe integer. Setelahnya, hasil deret ganjil ditampilkan ke layar menggunakan **print()**.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu15/soal3.py
Masukkan angka: 10
Hasil deret ganjil adalah 25
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu15/soal3.py
Masukkan angka: 9
Hasil deret ganjil adalah 25
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu15/soal3.py
Masukkan angka: 7
Hasil deret ganjil adalah 16
```

SOAL 4

Source code:

```
1 def hitung(angka):
2     if angka < 10:
3         return(angka)
4
5     return angka%10 + hitung(angka//10)
6
7 angka = int(input("Masukkan jumlah angka: "))
8 print("Jumlah digit adalah",hitung(angka))
```

Penjelasan:

Pada bagian awal program, dibuat fungsi bernama **hitung** dengan parameter bernama **angka**. Program pertama-tama akan memeriksa apakah nilai **angka** kurang dari 10 menggunakan **if angka < 10**. Jika kondisi ini terpenuhi, berarti angka tersebut hanya memiliki satu digit sehingga langsung mengembalikan nilainya. Namun, jika angka lebih dari atau sama dengan 10, program akan mengambil digit terakhir menggunakan operasi modulus **angka % 10**, kemudian menambahkan dengan hasil pemanggilan fungsi **hitung(angka // 10)** untuk memproses sisa satu digitnya. Proses ini berlangsung terus-menerus hingga seluruh digit habis dijumlahkan. Pada bagian akhir program, pengguna diminta memasukkan sebuah angka melalui **input()**, lalu diubah menjadi integer. Setelah itu, hasil penjumlahan digit akan ditampilkan ke layar menggunakan **print()**.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu15/soal4.py
Masukkan jumlah angka: 234
Jumlah digit adalah 9
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu15/soal4.py
Masukkan jumlah angka: 10000
Jumlah digit adalah 1
```

SOAL 5

Source code:

```
1 def kombinasi(n,r):
2     if r == 0 or r == n:
3         return 1
4
5     return kombinasi(n-1,r-1) + kombinasi(n-1,r)
6
7 n = int(input("Masukkan n: "))
8 r = int(input("Masukkan r: "))
9
10 print("Hasil kombiasi adalah", kombinasi(n,r))
```

Penjelasan:

Pada bagian awal program, dibuat sebuah fungsi bernama **kombinasi** menggunakan dua parameter yaitu **n** dan **r**. Parameter **n** sebagai jumlah objek dan parameter **r** menunjukkan jumlah objek yang dipilih. Program pertama-tama memeriksa apakah kondisi dasar menggunakan **if r == 0 or r == n**. Jika salah satu kondisi terpenuhi maka fungsi akan mengembalikan nilai 1. Jika kondisi tersebut tidak terpenuhi, maka program akan menjalankan **kombinasi(n-1, r-1) + kombinasi(n-1, r)**. Proses ini akan terus berjalan hingga mencapai base case. Pada bagian akhir, pengguna diminta untuk memasukkan nilai **n** dan **r** melalui **input()**, kemudian keduanya akan diubah menjadi integer. Setelah itu, kombinasi ditampilkan ke layar menggunakan **print()**.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu15/soal5.py
Masukkan n: 15
Masukkan r: 15
Hasil kombinasi adalah 1
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu15/soal5.py
Masukkan n: 15
Masukkan r: 13
Hasil kombinasi adalah 105
```